



Figure 1: ERD-Employee, Project, Customer

	A	B	C	D
1	Date	Project	Customer	Hours
2	01-Feb-15	ProjectA	Jackie Chan	10

Figure 2: Sample data - test.xlsx

pivot tables inside a master Excel file. Instead, let us look at a different way of doing the same task, using Python and Excel interfacing libraries.

Introducing the Python-Excel interfacing library

Let me introduce the *xlrd* package for Python available from the Python Foundation at <https://pypi.python.org/pypi/xlrd>

The package used to write into Excel files is available as *xlwt*. As a collective package, *xlutils* comprises both *xlrd* and *xlwt*. In this column, we use only *xlrd*.

xlrd is a library that can be used to work with Microsoft Excel spreadsheets (*xls* and *xlsx*) on any platform. Let us check the interfaces available in the following sections and look at how to use them.

Installation of the package

The installation instructions in the *readme.html* file are pretty clear. On any OS, unzip the *.zip* file into a suitable directory, *cd*dir to that directory, and type in *python setup.py install*.

If the installation is successful, you should be able to do what's shown below at the IDLE GUI prompt:

```
>>> import xlrd
>>>
```

Important interfaces of the *xlrd* package

xlrd provides the following interfaces for reading the Excel values.

1. *xlrd:open_workbook(path_to_Excel_file)*: This method returns an object of the *Book* class, with which we will work quite a bit to get access to the Excel data.
2. *Book:sheet_by_index(sheetx)*: This returns an object of the *Sheet* class, with which we will access the individual cells of a sheet.
3. *Sheet:cell(row,col)*: This returns a *Cell* object for the particular row and column values.
4. A *Cell* object should always be retrieved from other methods described above from the *Book* and *Sheet* objects and not invoked directly. Critical for a *Cell* object is the value attribute, the type of which varies according to the data type of the cell represented by

another *int* attribute which is *c_type*. The following table gives the type that value has, which depends upon the type of the cell.

Type symbol	Type number	Python value
XL_CELL_EMPTY	0	empty string ""
XL_CELL_TEXT	1	a Unicode string
XL_CELL_NUMBER	2	Float
XL_CELL_DATE	3	Float
XL_CELL_BOOLEAN	4	int; 1 means TRUE, 0 means FALSE
XL_CELL_ERROR	5	int representing internal Excel codes; for a text representation, refer to the <i>error_text_from_code</i>
XL_CELL_BLANK	6	empty string "". Note: this type will appear only when <i>open_workbook(..., formatting_info=True)</i> is used.



Note: In this column, we will work mostly with text and date types only.

Working with dates

As quoted in the *xlrd* tutorial, in Excel, dates have many problems.

1. Dates are not stored as a separate data type but as floating point numbers, and you have to rely on (a) the 'number format' applied to them in Excel and/or (b) knowing which cells are supposed to have dates in them.
2. Dates are stored as elapsed days from epoch, which differs between Windows and Mac versions of Excel.
3. The Excel implementation of the default Windows 1900 based date system works on the incorrect premise that 1900 was a leap year.



A word of caution

This method throws errors in case the data is not a date type or if there are other issues with the data. These need to be handled in the code, which must smoothly process such erroneous conditions.

Sample usage

Let us now put together a small spreadsheet with the data as shown in Figure 2.

We can open this using the interfaces of *xlrd* in the IDLE GUI.

```
>>> testbook=xlrd.open_workbook('C:\\test.xlsx')
>>> testbook
```